Telescope — Zoom into your graph

Visual Analytics for Non-Expert Knowledge Graph Exploration

Source code on GitHub

Francesco Secoli

June 4, 2026

Contents

Acknowledgment

The design is mine: the architecture, the interaction model (the bitmap-truth contract, mask-only propagation, the panel system), and the analytical goals. I used Claude Code (Anthropic's CLI) for the implementation across the stack (Vue, D3, FastAPI, NetworkX, NetworkKit), under step-by-step supervision. The usual loop: I wrote a short `.md` spec and a code skeleton, Claude filled it in, and I reviewed the result; I checked "it works" claims by driving the running app headlessly (Playwright) and gated frontend changes on a clean Vite build + ESLint. Bug fixes and documentation were the only steps it ran on its own, and I checked those too. Every design decision and final acceptance is mine.

1 Introduction

Telescope is a web-based visual-analytics *prototype* for the first look at an attributed knowledge graph, in the early *data-understanding* phase. Once a graph is loaded, it computes the main metrics and shows them as interactive panels: no notebook, no NetworkX code. As a prototype it aims at breadth, not production polish, so a few rough edges are expected (some are flagged later). This report covers the part built for this project: the dataset upload and loading path, and the *Guide view*, its interactive interface. It is designed for three use cases:

- **Accelerated verification**: when the analyst already has a question (*do `NodeType` nodes act as hubs?*, *is the degree heavy-tailed?*), the right panel turns it into an answer in a few clicks.
- **Cued discovery**: when the analyst has no direction yet, the panels draw the eye through preattentive pop-out (*a giant component dominating the bubble pack*, *a degree bar far out in the tail*, *a type-mixing cell coloured where no edge should exist*).
- **Learning by exploration**: reading a graph next to each panel's theory drawer builds understanding of both the loaded graph and knowledge-graph metrics in general.

Getting started

Download or pull the source from GitHub, then:

1. Run `./dev.sh`: it installs the dependencies and starts the backend and frontend (needs a Linux shell with `bash`, `python3`, and Node). Optional flags: `--dev` also runs the API tests, `--log` shows the full install output.
2. Open the `localhost` URL it prints.
3. Load a built-in graph or **upload** a NetworkX node-link JSON file.
4. Either path leads to the **Guide view**, where the analysis happens.

1.1 Datasets

Besides user uploads, the dashboard ships a few built-in datasets: a way to get familiar with the interface and a ready set of base cases for anyone learning to read these metrics. When a graph has no explicit type, it either *auto-promotes* a splitting attribute to the effective type or leaves it untyped; **Table 1** shows what the user sees by default (**Section 2.2**).

	Provenance		Structure						
Dataset	**Source**	**License**	**Nodes**	**Edges**	**Directed**	**Weighted**	**Bipartite**	**Node types**	**Edge types**
Karate	NetworkX	BSD-3	34	78		✓		2	—
Les Misérables	NetworkX	BSD-3	77	254		✓		—	—
Florentine	NetworkX	BSD-3	15	20				—	—
Davis	NetworkX	BSD-3	32	89			✓	2	—
Email-EU-Core	SNAP	BSD	1,005	25,571	✓			42	1
MovieLens	GroupLens	CC BY 4.0	10,334	100,836		✓	✓	2	1
Reference graph, uploaded (not a built-in):									
MC1	VAST 2025	—	17,412	37,857	✓			5	12

Table 1: Built-in datasets and MC1 (challenge's reference graph) structural profile (✓ = yes, — = untyped scope).

2 Design and Method

The dashboard is *read-only*: it inspects and explores a graph, it does not build or edit one. A few principles guided its design[1]:

- **One view per question:** never draw the whole graph as one tangled node-link picture. Each panel answers one metric clearly, and node-level detail is one drill-down away.
- **Progressive disclosure:** show a rich chart by default, keep its controls in a drawer, and add an interactive theory drawer that explains the metric in plain language, so depth is there without crowding the card.
- **Attenuate, don't recompute:** a filter dims the marks it removes; it does not recompute the metric on the surviving subset. So a subset is always read against the whole graph (the mechanism, and the few views that do recompute, in **Section 2.2**).
- **Colour by data type:** the kind of value sets the scheme: categories get distinct colours (and the same one in every panel), a count a light-to-dark scale, a signed correlation a blue-to-red one. When the types outnumber the easily distinct colours, node types also take a shape, so colour is not the only cue.
- **Real structure, no synthetic types:** show the graph as it is. If one type splits along an attribute, use that attribute as the type; if nothing splits it, leave it untyped (**Section 2.2**).

Because the target is *any* suitable knowledge graph, these choices have been validated across the built-in graphs.

2.1 System and Interface

The interface adapts to the loaded graph: a schema endpoint inspects it and shows only the panels that apply, with structural badges ("directed", "weighted", "bipartite"). A **panel registry** is the single source of truth for the Guide grid.

The panels form one linked workspace through three separate interaction channels:

- **Filtering** (global): narrows the data and dims the marks left out. Set from the sidebar or by interacting with a plot (*brushing the degree histogram, clicking a component*), it propagates to every panel.
- **Selection** (global): marks nodes or edges and highlights them across all views. Made by clicking marks; like filtering, written once and read everywhere.
- **Controls** (local): a panel's own (*log scale, top-N, bin size, view toggle*); they change that panel only, touching no shared state.

A single left sidebar is the control surface (**Figure 1**). It shows the graph's identity and a Contents/-Filters pill: Contents lists the panels, Filters opens the filter editor. The editor narrows the graph by type, by degree or weight range, and by per-type attributes, with undo, redo, and reset-all on top. Every edit flows through the shared bitmaps of **Section 2.2**.

Every panel card carries the same header icons, in order: open its **controls** drawer, **lock** the panel to freeze its view (**Section 2.2**), **maximise** it, open the **focus** view, and **remove** it.

[1] Given the prototype's size, not every case has been tested, so these are aims the design follows, not properties it always reaches.

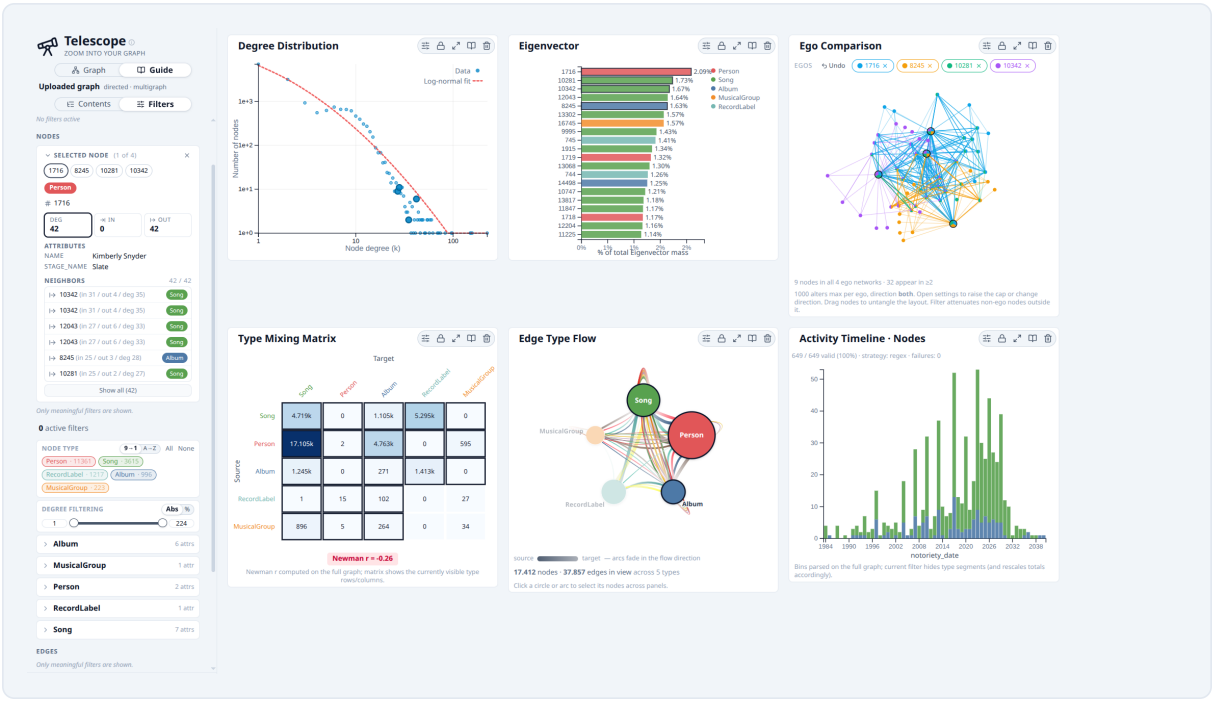


Figure 1: The dashboard's Guide view on MC1: the left sidebar (mode toggle, node inspector, type chips and structural filters) and a grid of metric panels (Degree Distribution, Eigenvector, Ego Comparison, Type Mixing, Edge Flow, Activity Timeline). A type keeps the same colour across every panel.

2.2 Cross-Panel Contracts

The naive design lets each panel hold its own filtered copy of the data: a dozen panels, a dozen states to keep in step, and the moment two disagree the views silently diverge. Telescope instead turns the filters and the selection, once, into three shared masks that every panel reads. Filtering happens a single time, and two panels cannot disagree because there is only one state to read. Five contracts make this hold, specified in full in `docs/sclfnc/contract.md`; this section gives the design intent, not the mechanics.

1. **Bitmap-truth:** the filters and the selection become three shared masks (which nodes pass, which edges pass, which are selected). Every panel reads the masks, never the raw filters: the one place where state lives.
2. **Mask-only:** a filter fades the marks it removes but does not recompute the metric, so each chart keeps the whole graph as a reference. The two count views (Type Mixing, Edge Flow) are the deliberate exceptions, since watching the count change is their point.
3. **Selection:** one shared list of picked nodes or edges; clicking a mark in one panel highlights it in every other.
4. **Effective-type:** when a graph has a single type but a splitting attribute (Karate's `club`, Davis' `bipartite`), that attribute becomes the type everywhere: colours, legends, groupings, chips.
5. **Lock (Isolation):** a panel can be frozen on its current view, keeping a snapshot of the masks until it is unlocked (an amber ring marks it).

2.3 A look at the client–server load

When a graph is opened, the dashboard loads everything the panels need at once: the nodes and edges, the schema, the attribute index, and each panel's data (degree fit, components, type mixing, edge flow, the timeline, and the four centralities). Anything tied to a single node is loaded only on demand: the ego view and the node and edge inspectors. Each result is computed once and cached, so the work is not repeated. The centralities are the slowest to compute, so they run in the background and stream in as they finish, and the page never waits for them; the large responses are gzip-compressed on the way out.

Table 2 shows this first-load data. *Raw/Source* is how much bigger the computed data is than the input file (usually more than 1, because the dashboard adds the centralities, fits, and indices). *GZip/Source* is how much actually travels over the network: much smaller than the input on the big graphs (0.30× on MC1, 0.18× on MovieLens), and about the same size only on the tiny ones.

	Structure		Payload			Ratio to source	
Graph	**Nodes**	**Edges**	**Source**	**Raw**	**GZip**	**Raw**	**GZip**
Florentine	15	20	1.3 KB	15 KB	3 KB	12.0×	2.7×
Karate	34	78	4.3 KB	30 KB	5 KB	7.0×	1.2×
Davis	32	89	5.7 KB	31 KB	6 KB	5.4×	1.0×
Les Misérables	77	254	16 KB	67 KB	10 KB	4.1×	0.6×
Email-EU	1,005	25,571	1.36 MB	1.70 MB	464 KB	1.2×	0.33×
MC1	17,412	37,857	6.22 MB	14.91 MB	1.86 MB	2.4×	0.30×
MovieLens	10,334	100,836	10.97 MB	10.07 MB	1.96 MB	0.9×	0.18×

Table 2: First-load payload per graph (on-demand `/ego/`, `/node-inspect/`, `/edge-inspect/` calls excluded). *Source* is the input JSON; *Raw*/*GZip* sum the first-load endpoints, each divided by *Source*.

3 The Panels, in Theory and in Use

Thirteen panels are built. Each draws a chart in the grid and adds an interactive theory drawer in its focus view. **Table 3** gives one line per panel: the attribute it shows, whether that attribute is a category, an order, or a number, and the visual channel used for it. The rule, from **Section 2**, is simple: numbers go on position or length, categories on colour, and any other choice is made on purpose.

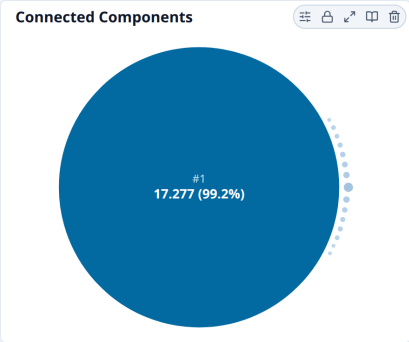
Panel	Encoded attribute	Type	Channel
Descriptive Metrics			
Degree Distribution	Degree; fitted tail model	number; category	position (log), length; hue
Connected Components	Component size	number	area *or* length
Centrality			
Centrality (×4)	Centrality value; node type	number; category	position/length; hue
Centrality Comparison	Pairwise correlation	signed number	diverging colour + position
Mixing & Assortativity			
Type Mixing Matrix	Edges per type pair	number	value/intensity (sequential)
Edge Flow	Flow per type pair; edge type	number; category	arc width; hue + gradient
Local Structure			
Ego Network	Adjacency; node type; direction	–; category	node-link position; hue; arrowheads
Ego Comparison	Ego membership of shared nodes	category	pie wedges (angle)
Temporal Analysis			
Activity Timeline	Time; per-bin count; type	number; category	position (x); length; hue

Table 3: The panel catalogue at a glance, grouped by sidebar section: encoded attribute, its data type, and the channel chosen for it. The four Centrality panels share a row (same encoding, different statistic).

3.1 Encoding choices worth a note

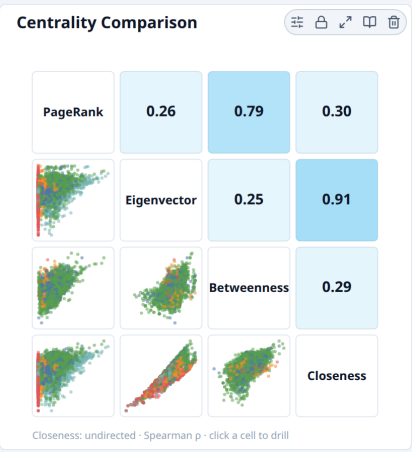
Three panels make an encoding choice that departs from the default rule, each on purpose.

Connected Components: accuracy versus immediacy



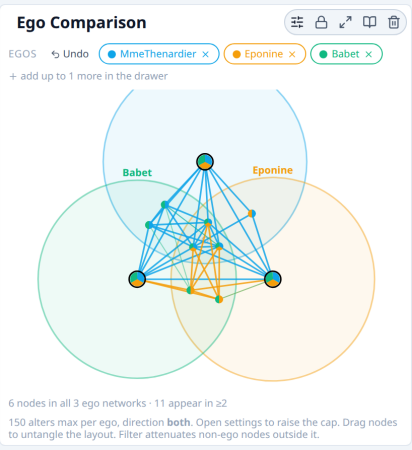
Shown on MC1 in its bubble view. The size of a component is a number, and bars would encode it on length, the channel the eye reads most accurately. This panel also offers bubbles, which encode size on area instead: less precise to compare, but the packed layout answers the first question, is the graph one piece or many, at a glance. Both views are available and the user switches between them. This is the one place the dashboard trades some accuracy for an immediate overview.

Centrality Comparison: a colour scale for agreement and disagreement



Shown on MC1. The matrix compares the four centrality measures two at a time: mini-scatters below the diagonal, the correlation between them above it. That number is signed, so the colour scale runs blue to red with white at zero: blue for directly proportional, red for inversely proportional, darker for a stronger link either way. Both ends mean a strong relationship ($+1$ and -1 alike); only a value near zero, pale here, means the two are unrelated, and a plain light-to-dark scale would have hidden that sign.

Ego Comparison: angle as a category, on purpose

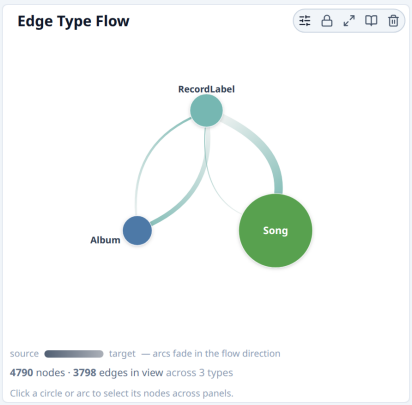


Shown on Les Misérables. Up to four chosen nodes (the egos) are drawn as circles, and a node that several of them share sits in the overlap, drawn as a small pie. Here the slices of the pie do not measure a quantity: they name *which* egos contain that node. This is the one deliberate exception to the rule that angle is for numbers, because a Venn-style diagram is the natural picture for showing what belongs to which set.

3.2 A worked analytical walkthrough on MC1

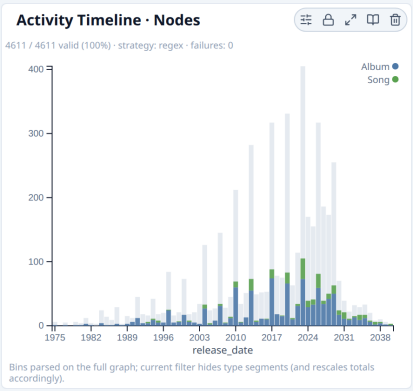
This section demonstrates the design on concrete data. Two walkthroughs work through MC1, the challenge's reference graph: an audit of a relation against its intended meaning, and the study of a trend over time. A final part then validates the same design across the other built-in graphs (**Table 4**). Each walkthrough follows the same path, *goal* → *visual* → *observation* → *verification* → *conclusion*, and every finding was first confirmed in NetworkX on the raw graph.

Audit: a reversed `RecordedBy`, seen in Type Mixing and Edge Flow



The *goal* is to check whether any relation points the wrong way. `RecordedBy` should run work → label, and 3,695 edges do; but **103 run the other way** (`RecordLabel`→`Album`, 102; `RecordLabel`→`Song`, 1). The *visual* is Type Mixing filtered on `RecordedBy`: the correct mass fills the `RecordLabel` column, while a cell lights up in its row, the *observation* that **the direction is reversed there**. Edge Flow *verifies* it by bowing the two senses to opposite sides, so the thin reversed flow reads apart from the dominant one. The *conclusion*: 103 reversed edges, while the twin relation `DistributedBy` has none, so this is a defect, not a property of the domain.

Discover: the rise of Oceanus Folk over time (Activity Timeline, `release_date`)



The *goal* is to tell whether `Oceanus Folk` rose gradually or in bursts. The *visual* is the Activity Timeline with the genre filtered on `Song` and `Album` (305 works) over `release_date`: the coloured bars are the genre, the grey silhouette the whole catalogue. The *observation* is a steady climb from the late 1990s that accelerates in the late 2010s, read against that baseline; *verified* by the masked bars holding the baseline's shape as a rising fraction of it. The *conclusion*: a gradual, accelerating rise, not a burst.

Graph	What it shows
Karate	auto-promotion on `club` colours the two factions, and Betweenness picks out their leaders (nodes 0 and 33): the most between-central nodes *are* the schism's protagonists.
Davis	Type Mixing and Edge Flow show the bipartite signature (no same-side edges); Ego Comparison separates the two social circles.
Florentine	with a single type and 15 nodes, pure centrality speaks: Betweenness ranks the Medici far above the rest, Padgett's classic reading of their brokerage.
Les Misérables	co-appearance weights and a clear degree hub (Valjean) drive the Degree Distribution and the weighted centralities.
Email-EU-Core	42 department types: the 42×42 Type Mixing is where pushing past the hue ceiling onto legend and filter pays off, answering the "small graphs only" objection.
MovieLens	analysed undirected, so betweenness and closeness stay meaningful; the Activity Timeline shows the rating wave and the Degree Distribution the long recommendation tail.

Table 4: A deliberate validation suite: each built-in graph stresses a different capability and surfaces one known, non-trivial result, one example per graph and not the only insight it admits.

3.3 Caveats

As a prototype, the dashboard favours a fast, readable answer over the most exact value on every possible graph. The points below are the trade-offs identified during design: a representative subset, meant to be explicit about what is known rather than exhaustive.

- **Degree Distribution** fits all four models (power-law, exponential, Poisson, log-normal) on the same lower bound. This makes the power-law fit conservative, but keeps the four likelihoods directly comparable.
- **Centrality (Eigenvector, Closeness)** is restricted to the principal component, where the measure is well-defined: Eigenvector on the largest component, Closeness per component. On a connected graph this has no effect.
- **Type Mixing** computes the Newman coefficient r on an undirected, simple projection, so on a directed multigraph it describes a slightly different object than the matrix beside it. The panel flags this.
- **Ego Network** refuses neighbourhoods above $\sim$2,500 nodes and samples dense egos in a stratified way. The cap is set high on purpose: it refuses only the genuinely unreadable, since a node-link layout is illegible well before that.
- **Connected Components** caps per-component detail at 200 records. Aggregate figures (count, giant-component fraction) stay exact.
- **Type Mixing and Edge Flow** cap a broadcast selection at 100 and 200 nodes; the rest is reported as a "$+N$ more" caption.
- **Activity Timeline** infers a parsing strategy per date field (user-overridable) and drops values it cannot parse, so irregular or free-text dates are absent.
- **Attribute filters** treat high-cardinality attributes (a `name`, an identifier) as free-text search rather than chips, at the risk of misclassifying a categorical attribute with very many levels.

4 Scope, Limitations, and Conclusion

Beyond the read-only stance and the per-panel simplifications already discussed, four boundaries define the scope of the prototype:

- **No persistence:** the registry lives in memory, so a restart clears the loaded graphs, acceptable for a single-user prototype with no accounts.
- **Single process:** all state (registry, caches, centrality tasks) is in-process, so the backend runs as one worker; concurrency and multi-process serving are out of scope.
- **Scale ceiling of about 100K edges:** on the datasets provided, the dashboard stays clearly responsive (**Table 2**), since the whole graph is sent to the browser, which does the filtering. A substantially larger graph would move that work back to the server, sending only the part the client needs, which is out of scope here.
- **No layout breadth:** it is not a Gephi replacement, but a tool for the *exploratory* phase that precedes a full layout or community workflow, when the analyst has not yet decided which metric matters.

Within those boundaries, the structure of an unfamiliar knowledge graph becomes legible through a set of single-purpose panels. Each shows one standard metric on the loaded data, pairs it with a plain-language theory drawer for the non-expert, and stays consistent with the others through the shared-bitmap contract, so the whole graph remains a stable reference under any filter or selection. The walkthrough on MC1 (**Section 3.2**) makes the case concretely, finding a reversed relation that no panel was specifically built to catch and reading a genre's trend over time, while **Table 4** shows the same panels, unchanged, rediscovering well-known results on five other graphs.